Centro Universitário de Excelência
Sistemas de Informação

MVP MecâniQA: Gestão de Peças e Serviços com Arrays Estáticos

Autor:
Caio Josué Ribeiro Bastos
Camilly Vitória Ribeiro Ferreira
Davi Merces da Silva
Filipe Maia Archanjo dos Santos de Santana
Jackson Souza de Souza
Julia Gomes de Souza

Feira de Santana, 2026

Agenda

1. **Contexto do problema: Desafio proposto pela MecâniQA e objetivos da sprint**

2. **Modelagem dos TADs: Estruturas Peça e Serviço, no padrão struct-like**

3. **Motor de gerenciamento: CRUD completo (Create, Read, Update, Delete) sobre arrays estáticos**

4. **Busca linear e atualização: Localização de registros e reuso de lógica (DRY)**

5. **Resultados e considerações: Testes de QA, limitações e próximos passos**

O objetivo desta apresentação é demonstrar como a nossa equipe modelou e implementou o módulo fundacional do sistema de gestão de estoque e serviços da MecâniQA, utilizando exclusivamente estruturas de dados estáticas em Java.

Contexto do Problema

- A MecâniQA precisa da primeira versão (MVP) do sistema interno de gestão de estoque e serviços — esta é a Fase 1: o módulo fundacional em arrays estáticos
- O núcleo deve gerenciar a memória de forma manual e contígua, usando unicamente arrays estáticos
- Peca[100] — capacidade máxima de 100 registros
- Servico[50] — capacidade máxima de 50 registros
- Objetivos da sprint: TADs, manipulação de arrays, busca linear e análise das limitações de estruturas estáticas
- Vantagem da alocação estática: dados ficam em posições contíguas na memória, reservadas de uma vez — isso é o que permite o acesso O(1) por índice

Modelagem dos TADs

- Itens para discussão:

● TAD (Tipo Abstrato de Dado): forma de representar e agrupar dados relacionados sob um mesmo tipo — aqui, implementado como struct, com apenas atributos e nenhum comportamento (método) interno

● As entidades foram criadas como structs do C: atributos públicos, sem getters/setters e sem construtores customizados

○ Peça: codigo (int), nome (String), fabricante (String), precoCusto (double), precoVenda (double), quantidadeEstoque (int)

○ Serviço: codigo (int), descricao (String), tempoEstimadoMinutos (double), valorMaoDeObra (double)

○ Decisão alinhada à diretriz do Diretor de Engenharia, priorizando simplicidade estrutural



Motor de Gerenciamento — CRUD

- Toda a lógica de negócio foi implementada por meio de métodos estáticos na classe Gerenciador

  - Create: inserirPeca / inserirServico — insere no primeiro espaço vazio, código gerado automaticamente

  - Read: listarPecas / listarServicos — exibe todos os registros ativos

  - Update: atualizarPeca / atualizarServico — localiza o item pela busca linear e sobrescreve os dados

  - Delete: removerPeca / removerServico — técnica de reorganização (shift), deslocando os elementos seguintes



Listas Estáticas: Índices e Custos

- Gestão de Índices: os contadores totalPecas e totalServicos sempre apontam para o próximo espaço vazio — dispensa percorrer o array para saber onde inserir

- Custo da Busca Linear: para localizar um item pelo código, é necessário percorrer o array sequencialmente, comparando posição por posição — custo O(n) no pior caso

- Operações Virtuais — custo da remoção, duas técnicas possíveis:

    - Exclusão lógica: marca o item como inativo sem mover nada — custo O(1), mas deixa "buracos" e exige checagem extra em toda leitura

    - Reorganização (shift): desloca os elementos seguintes para preencher o espaço — custo O(n) no pior caso, mas mantém o array sempre compacto

- A equipe optou por reorganização, priorizando simplicidade nas demais operações (listagem, busca) em troca de um custo maior apenas na remoção



Considerações finais

- Limitação estrutural: com o array em 100 posições, o sistema não aceita a 101ª peça — exigiria migração para estrutura dinâmica (ex: ArrayList) em versões futuras

- Ganho de performance inicial: memória contígua e alocação estática garantem previsibilidade e velocidade no MVP

- Testes de QA validados: QA-01 (inserção válida), QA-02 (array cheio), QA-03 (índice inválido) e QA-04 (atualização de código inexistente) — todos passaram

- Próximos passos: persistência em arquivo/banco de dados e substituição gradual dos arrays por coleções dinâmicas

Referências

● UNEX — Centro Universitário de Excelência. Guias das Sessões Tutoriais 1, 2 e 3. Feira de Santana, 2026.

● MecâniQA — Automotive Tech. Especificação do Produto Final — OAT 1: Explorando Estruturas de Dados Estáticas. Feira de Santana, 2026.

● Repositório do projeto: github.com/Davibesxxol/mecaniQA-natal